PDF PROCESSING – TECHNICAL TEST.

1. INSUMOS DE LA PRUEBA.

Para este desafío, se te proporcionan 5 archivos:

- **2 archivos PDF:** Los boletines originales (como referencia visual).
- **2 archivos JSON con sufijo 001**: contienen los elementos de cada PDF, extraídos con PDFPlumber. Contiene una lista de objetos, donde cada uno representa una página. Cada página tiene a su vez sub-elementos, siendo los más relevantes:
    - **page**: Número de página.
    - **textboxhorizontal**: bloques de texto.
    - **rects** y **line**: elementos que pueden servir para detectar líneas de separación.
    - A su vez los elementos pueden contener parámetros como:
        - text: Contenido textual para los bloques de texto
        - x0, x1: Posición horizontal (izquierda y derecha).
        - top, bottom: Posición vertical (distancia desde el borde superior).
- **1 archivo JSON con sufijo 002:** Es una muestra del esquema final que debe generar tu script.

2. OBJETIVO.

Tu misión es escribir un script en Python que procese el archivo **BUL_EM_TM_2024000007_001.json** para extraer exclusivamente los registros de la **Sección B.1.** y generar el archivo **BUL_EM_TM_2024000007_002.json.**

3. DESAFÍOS A RESOLVER (CRITERIOS DE EVALUACIÓN).

Debes demostrar cómo resuelves mediante código los siguientes problemas geométricos y lógicos:

- **Filtro de Sección:** El archivo fuente contiene todo el boletín. Tu script debe identificar dónde empieza y termina la sección **B.1** basándose en el contenido de los textos y su jerarquía.
- **Reconstrucción de Columnas:** Los datos en el PDF están dispuestos en dos columnas. Debes usar las coordenadas **x0** y **x1** para separar correctamente la información y evitar que el texto de la columna izquierda se mezcle con la derecha.

UNUMBIO SpA | PROPRIETARY & CONFIDENTIAL | WHEN APPLICABLE: © ALL RIGHTS RESERVED

- **Identificación de Registros:** Cada registro de marca comienza típicamente con un código INID (ej. el código **111**). Debes implementar una lógica que agrupe todos los elementos de texto pertenecientes a un mismo registro antes de pasar al siguiente.
- **Manejo de Multilinealidad y Saltos de Página:** Algunos registros pueden dividirse entre dos columnas o páginas. El script debe ser capaz de agrupar esa información para que el JSON de salida no tenga registros duplicados o incompletos.

4. REQUISITOS DE ENTREGA.

- Código Python limpio (siguiendo PEP 8).
- Uso de librerías estándar o de procesamiento de datos (ej. **json**). No es necesario usar librerías de PDF ya que el JSON de entrada ya tiene las coordenadas.
- Memoria descriptiva explicando los principales desafíos encontrados y las características de las soluciónes planteadas.
- Archivo **BUL_EM_TM_2024000007_002.json** resultante. Debe cumplir con la misma estructura de datos mostrada en el documento **BUL_EM_TM_2024000007_001.json**.

5. NOTAS SOBRE LA ESTRUCTURA DEL JSON DE SALIDA.

- Estructura del JSON de salida:
    - Debe ser un objeto con una key para cada "sección". En este caso solo se pide la sección "B".
    - Cada sección debe ser un objeto con una key para cada "sub-sección". En este caso solo se pide la sub-sección "1".
    - La sub-sección debe contener un listado de objetos, uno para cada registro encontrado.
    - El primer parámetro de la sección debe ser "_PAGE", e indica la primera página donde se vio el primer campo del registro. La mayoría de los registros tienen todos sus campos dentro de una misma página, pero en algunos casos los registros pueden continuar en la siguiente página. En estos casos "_PAGE" debe ser la página donde empezó el registro.
    - "_PAGE" debe ser tipo "int", y el campo "400" debe guardarse como un listado de str, teniendo un elemento por cada línea presente. El resto de campos deben ser guardados como "str". Si un campo de texto tiene varios elementos de texto, entonces deben unificar en un solo elemento, uniendo las sub-partes con un espacio de por medio (" ").
- Los archivos se pueden entregar adjuntos por correo o como un link a un repositorio de Github.

UNUMBIO SpA | PROPRIETARY & CONFIDENTIAL | WHEN APPLICABLE: © ALL RIGHTS RESERVED

- En caso de no lograr culminar el objetivo en el tiempo estipulado, igual puedes enviar la memoria descriptiva, explicando los pasos desarrollados y los que te faltaron por hacer. Puedes crear un seudo-código o un template del código, especificando las funcionalidades que quedaron pendientes por desarrollar.

UNUMBIO SpA | PROPRIETARY & CONFIDENTIAL | WHEN APPLICABLE: © ALL RIGHTS RESERVED